PROJECT DOCUMENTATION

INTRODUCTION

Table of Contents

Online Quiz Application

A Django-based Timed Quiz Platform with Instant Scoring

1. Project Overview

Purpose

Online Quiz Application is a Django web app that lets a visitor register, log in, and take a timed multiple-choice quiz on core Python concepts. As soon as the quiz is submitted — either by the user clicking Submit or by the countdown timer running out — the app grades every answer server-side and shows a result page with the total score and a breakdown of exactly which questions were answered incorrectly.

The project demonstrates a complete, minimal Django auth-and-assessment flow: a public landing page, Django-Forms-based registration and login, a countdown-timed quiz screen, and instant, transparent grading.

Goals

- Provide simple, working registration and login using Django's User model and Django Forms.
- Present a fixed bank of multiple-choice questions with a visible countdown timer.
- Grade the quiz automatically on submission and clearly show which answers were wrong and what the correct answers were.
- Keep the project small and readable enough to serve as a college/portfolio project.

Features

1. Landing Page

A home page welcomes the visitor and offers Register and Login actions.

2. Django-Forms-Based Registration & Login

Registration uses a ModelForm bound to Django's built-in User model (username, email, password). Login uses a plain Django Form validated against authenticate().

3. Timed Quiz

The quiz page displays a live countdown ('Time Left: ... Seconds') alongside ten multiple-choice questions, each with four radio-button options.

4. Instant Result with Wrong-Answer Breakdown

On submission, the app displays the score out of ten and lists every question that was answered incorrectly, showing the submitted answer next to the correct one.

2. Architecture

Frontend — Django Templates

Technology: Django's server-side template engine, plain HTML/CSS, and a small amount of JavaScript.

Role: Renders the home, registration, login, quiz, and result pages.

Features:

- static/css/style.css provides shared page styling.
- static/js/script.js drives the live countdown timer and a confirmation dialog shown before the quiz is submitted.
- Templates: home.html, register.html, login.html, quiz.html, result.html.

Backend — Django Views & Forms

Technology: Django (Python) — forms.py, models.py, views.py, urls.py, admin.py.

Role: Handles authentication, quiz rendering, and grading.

Features:

- User authentication via django.contrib.auth (authenticate, login).
- Quiz questions stored as Question rows and served to the template on GET /quiz/.
- On POST /quiz/, every submitted answer is compared against the stored correct option and the result template is rendered directly with the score and the list of wrong answers.

Database — SQLite via Django ORM

Technology: SQLite (Django's default database).

Role: Stores registered users (auth_user) and the Question bank used to build the quiz.

Features:

- Question model holds the question text, four options, and the correct option.
- Questions are managed through the Django admin site.

3. Setup Instructions

Prerequisites

Dependency	Purpose
Python 3.12	Base programming language for the project
Django	Web framework providing ORM, auth, forms, templating
SQLite (bundled)	Default database engine, no separate install needed
pip	Installs Python dependencies
Virtual environment	Isolates project dependencies

4. Installation

Step 1: Clone the Repository

```
git clone https://github.com/your-username/onlinequiz.git
cd onlinequiz
```

Step 2: Create a Virtual Environment

```
python -m venv venv
source venv/bin/activate    # Linux/macOS
venv\Scripts\activate       # Windows
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Apply Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Step 5: Create a Superuser (to add quiz questions via /admin/)

```
python manage.py createsuperuser
```

Step 6: Run the Application

```
python manage.py runserver
```

Note: Add Question records through /admin/ before visiting /quiz/, or the quiz page will have nothing to display.

5. Folder Structure

The project follows the standard Django layout: a project-configuration package (onlinequiz) and a single app (quiz) containing the actual functionality.

Project — onlinequiz/onlinequiz/

Purpose: Django project configuration — settings, root URL conf, and ASGI/WSGI entry points.

```
onlinequiz/
|-- onlinequiz/
|    |-- __init__.py
|    |-- asgi.py
|    |-- settings.py      # INSTALLED_APPS, DATABASES, TEMPLATES config
|    |-- urls.py          # Root URL conf, includes quiz.urls
|    |-- wsgi.py
|
|-- manage.py
|-- db.sqlite3
```

App — quiz/

Purpose: Contains the forms, models, views, static assets, and templates for the whole application.

```
quiz/
|-- __init__.py
|-- admin.py          # Registers Question with the Django admin
|-- apps.py
|-- forms.py          # RegisterForm, LoginForm
|-- models.py         # Question model
|-- urls.py           # App-level URL routes
|-- views.py          # home, register, user_login, quiz
|-- migrations/
|
|-- static/
|    |-- css/
|    |    |-- style.css
|    |-- js/
|         |-- script.js    # Countdown timer logic
|
|-- templates/
     |-- home.html
     |-- register.html
     |-- login.html
     |-- quiz.html
     |-- result.html
```

6. Running the Application

Run the Development Server

```
python manage.py runserver
```

Then open the following URLs in a browser:

- http://127.0.0.1:8000/ — Home page (Register / Login)
- http://127.0.0.1:8000/register/ — create a new account
- http://127.0.0.1:8000/login/ — sign in
- http://127.0.0.1:8000/quiz/ — take the timed quiz and view the result
- http://127.0.0.1:8000/admin/ — add/edit quiz questions

7. View & URL Documentation

This is a server-rendered Django application, so each 'endpoint' is a URL route backed by a view that renders a template or redirects — there is no JSON API.

1. Home

GET /

Description: Public landing page with a welcome message and Register / Login buttons.

2. Registration

GET/POST /register/

Description: Registers a new user with RegisterForm, a ModelForm bound to Django's User model.

Form fields (POST):

```
username: 'navya'
email: 'navya@example.com'
password: '********'
```

Behavior: On valid submission a new User row is created and the browser is redirected to /login/. Field errors (including Django's default username help text) are shown inline on invalid input.

3. Login

GET/POST /login/

Description: Authenticates a user with LoginForm, then calls Django's authenticate() and login().

Form fields (POST):

```
username: 'navya'
password: '********'
```

Behavior: On success the user is logged in and redirected to /quiz/. On failure, login.html is re-rendered with an 'Invalid Username or Password' message.

4. Quiz & Result

GET/POST /quiz/

Description: GET renders quiz.html with all ten Question records and starts the client-side countdown timer. POST grades the submission and renders result.html directly (no separate /result/ route or redirect) — the browser tab and content change to the result view while the URL stays /quiz/.

Form fields (POST, one per question):

```
q1: 'print()'
q2: 'def'
q3: 'for'
...
q10: 'Pandas'
```

Behavior: Each submitted value is compared with Question.correct_option; the total score (e.g. 4/10) and the list of incorrectly answered questions — with the submitted answer and the correct answer — are passed straight into the result.html context.

8. Authentication

The application relies on Django's built-in authentication framework rather than a custom token scheme.

Authentication Method

Session-Based Login (Username & Password)

- Users register with RegisterForm; Django hashes the password (PBKDF2) before saving the User row.
- On login, LoginForm collects credentials and passes them to django.contrib.auth.authenticate().
- On success, django.contrib.auth.login() attaches the user to the request and Django issues a session cookie.

Session Handling

- Django stores session state server-side (database-backed sessions by default) and sends only a signed session ID to the browser.
- Because grading happens and renders the result in the same request/response cycle, quiz results do not need to be stored in the session.
- Sessions expire based on SESSION_COOKIE_AGE in settings.py, or on browser close if SESSION_EXPIRE_AT_BROWSER_CLOSE is enabled.

Request Protection

- All POST forms include {% csrf_token %}; Django's CSRF middleware rejects any POST without a valid token.
- The quiz view can be protected with @login_required so only authenticated users can take the quiz.

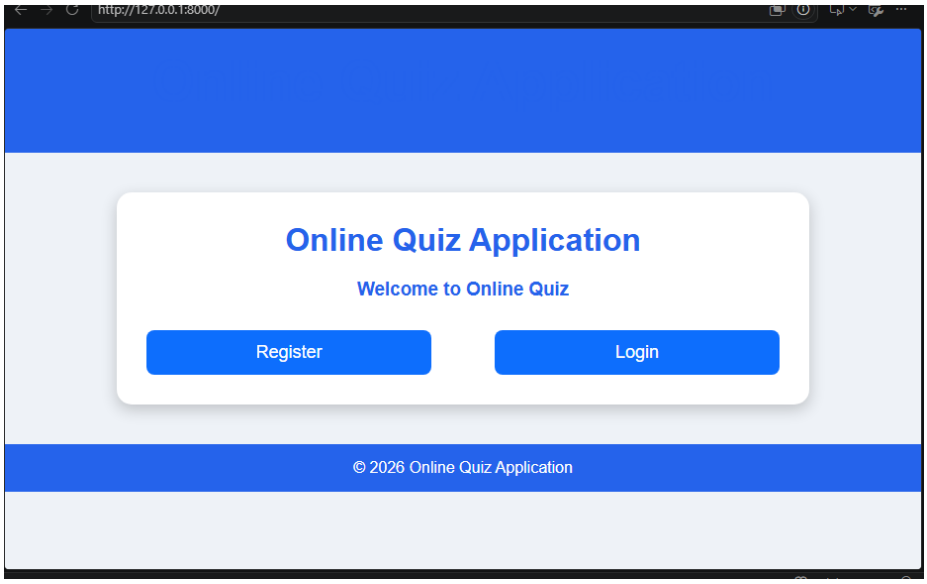
```
from django.contrib.auth.decorators import login_required

@login_required
def quiz(request):
    ...
```

9. User Interface

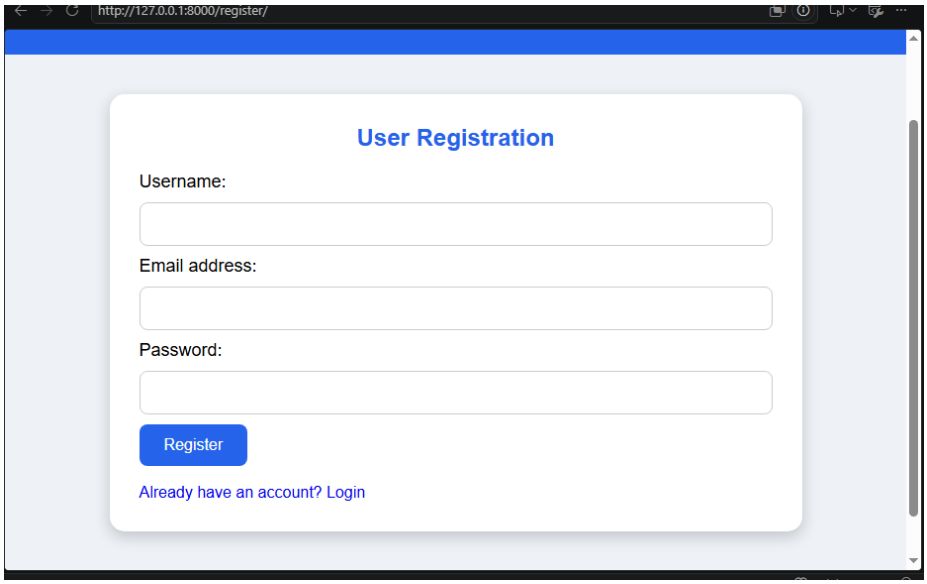
The screenshots below were captured from the running application.

Home Page — /



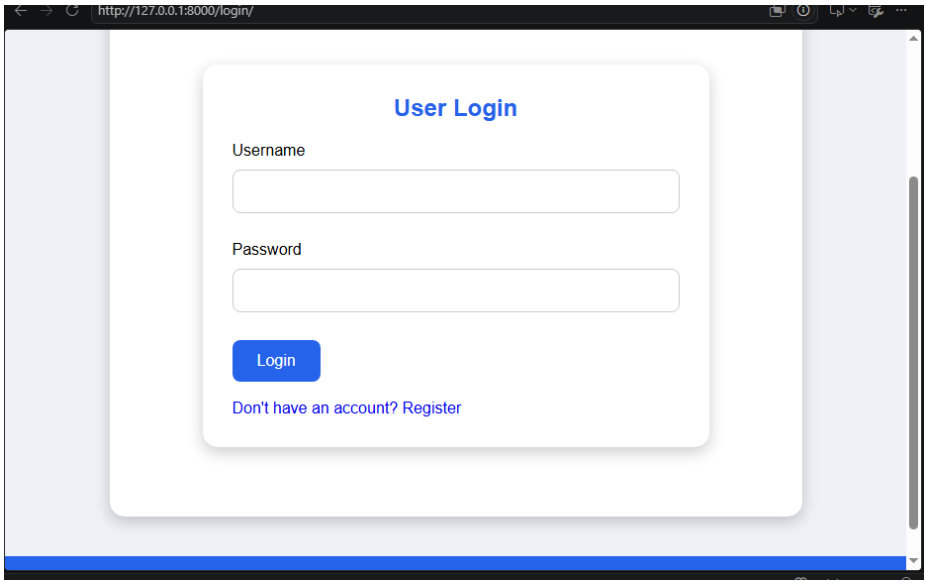
Home page with Register and Login actions.

Registration Page — /register/



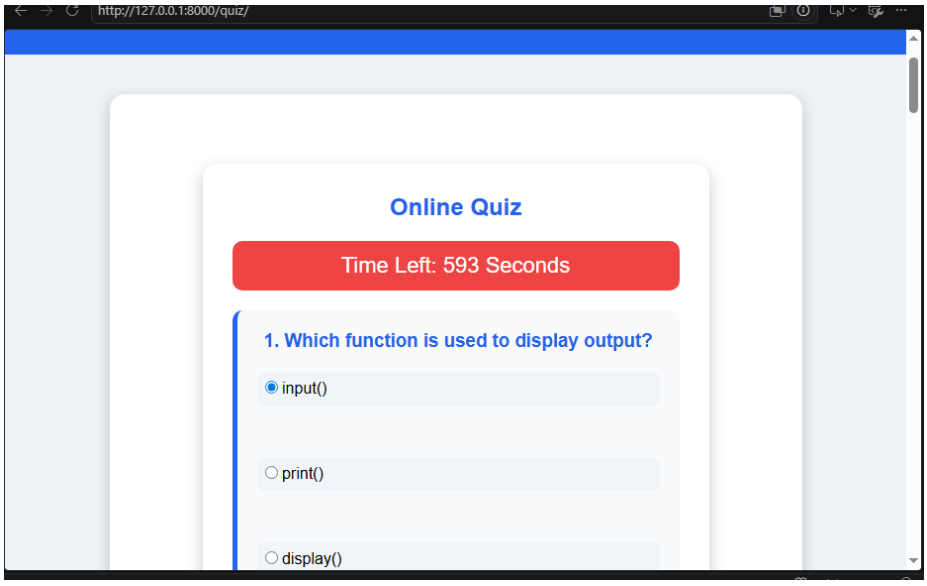
User Registration form (username, email address, password).

Login Page — /login/

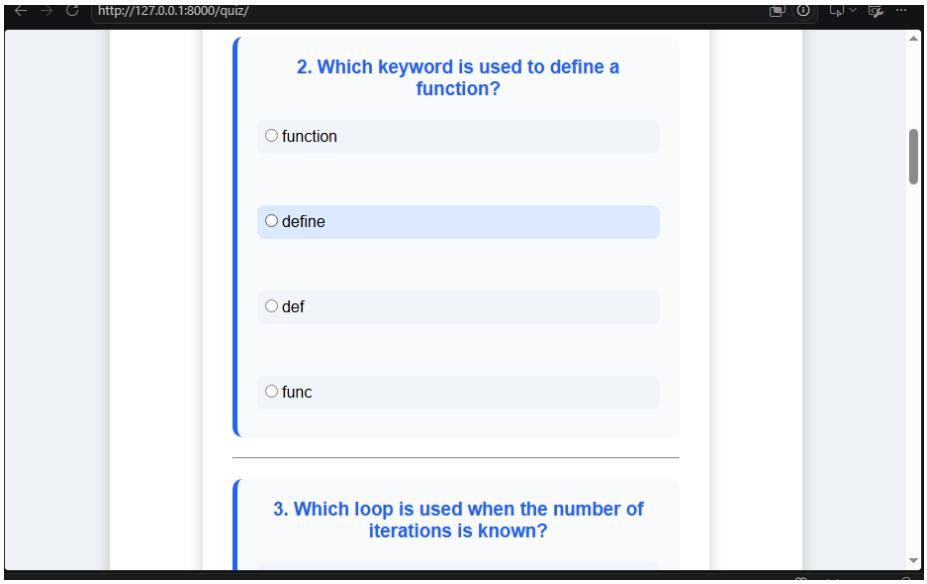


User Login form (username, password).

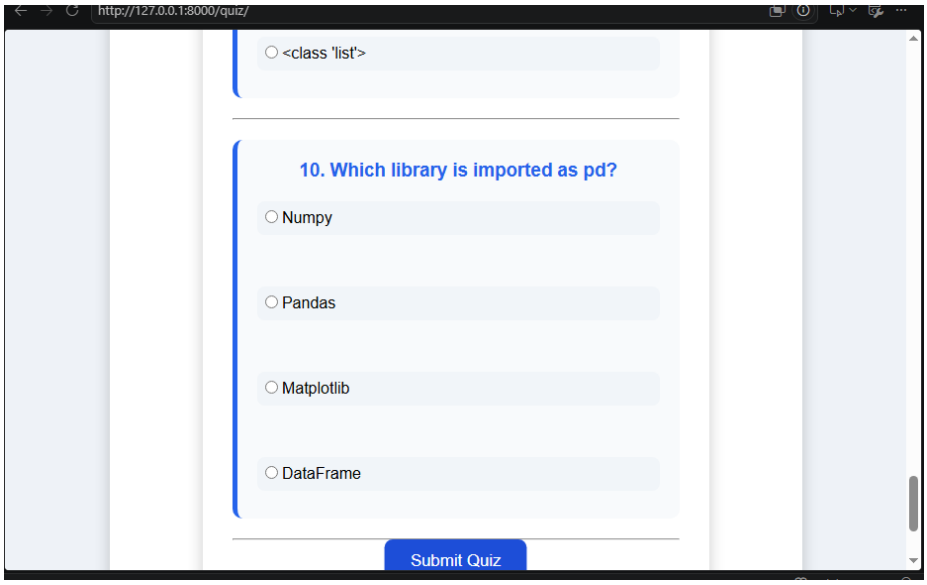
Quiz Page — /quiz/ (GET)



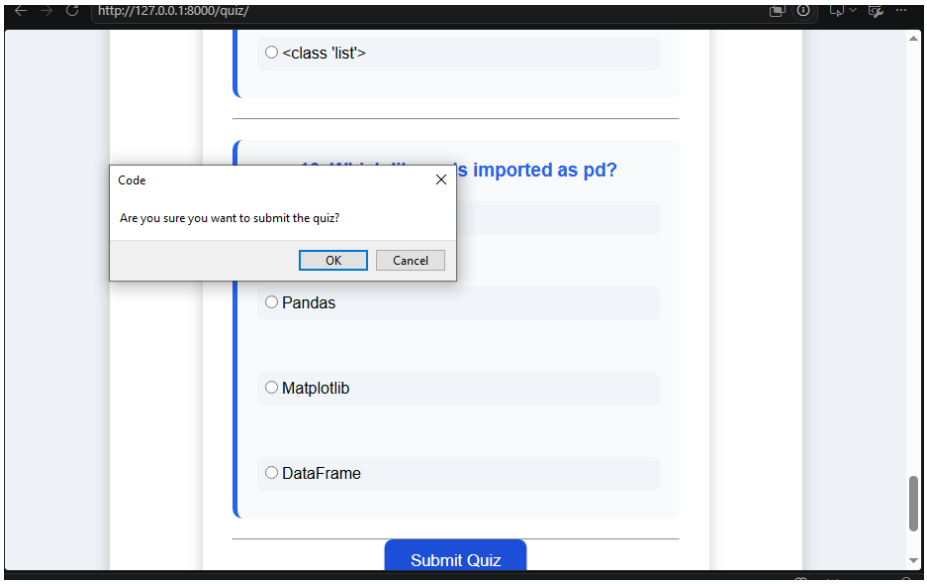
Quiz start: live countdown timer and question 1 of 10.



Scrolling down: questions 2 and 3.

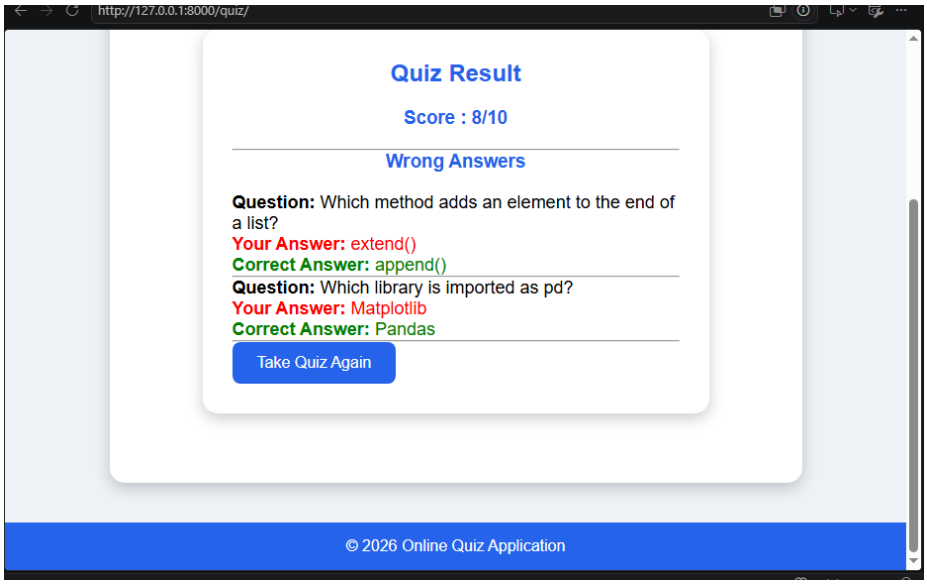


Final question (10) and the Submit Quiz button.



A confirmation dialog guards against accidental submission.

Result Page — /quiz/ (POST)



Quiz Result: score out of 10 and a list of wrong answers with the submitted vs. correct option.

10. Testing

Manual and lightweight automated testing was used to confirm the registration, login, and grading logic behave correctly.

Testing Goals

- Validate core functionality: registration, login, quiz rendering, and grading.
- Confirm invalid credentials and invalid form input are handled gracefully.
- Verify the score and the wrong-answer list always match the stored correct_option values.
- Confirm the countdown timer visually updates and that submission still grades correctly when time runs out.

Tools Used

Tool/Library	Purpose
Django TestCase	Unit and functional testing for views, forms, and models
Django Test Client	Simulates GET/POST requests against views in tests
Django Admin	Manual verification of Question data entry
Browser DevTools	Manual inspection of the countdown timer and form submissions

Types of Testing Performed

1. Unit Testing

Focused on isolated logic such as RegisterForm/LoginForm validation and the answer-comparison logic used for grading.

2. View / Functional Testing

- Successful and failed registration submissions
- Successful and failed login attempts
- Quiz submission with a mix of correct and incorrect answers, checking the resulting score and wrong-answer list

3. Manual UI Testing

Confirmed the templates render correctly in the browser, the countdown timer counts down as expected, and the result page correctly highlights wrong answers in red and correct answers in green.

11. Known Issues

1. Only Wrong Answers Are Shown

The result page currently lists only the questions answered incorrectly. It does not show a full question-by-question breakdown including correctly answered questions.

2. No Attempt History

Quiz attempts are graded and displayed once; they are not persisted to the database, so there is no record of past scores.

3. Timer Is Client-Side Only

The countdown timer in script.js is a visual/UX feature; there is no server-side enforcement that automatically submits or rejects a late submission if the client clock is tampered with.

4. Fixed Question Set

All users currently see the same ten questions in the same order, since there is no randomization or per-user question pool.

12. Future Enhancements

1. Full Answer Breakdown

Extend the result page to show every question — not just the wrong ones — so students can review what they got right as well.

2. Persist Quiz Attempts

Introduce a QuizAttempt model to store each submission (user, score, timestamp) for historical tracking.

3. Server-Side Timer Enforcement

Record a quiz start timestamp server-side and reject or auto-grade submissions that arrive after the allotted time, independent of the client-side timer.

4. Randomized Questions

Shuffle question order and answer-option order per attempt to reduce copying between users.

5. Polished UI

Apply a more distinctive visual theme to the home, registration, login, quiz, and result pages beyond the current style.css baseline.

6. Category / Difficulty Tagging

Extend the Question model with category and difficulty fields to support topic-based quizzes beyond the current fixed Python-basics

Prepared by:

P. Chaitanya Deepthi